

Growth Management System (GMS)

Technical Architecture – MVC, APIs, Routes & Database Design

1. Architecture Overview

The system follows **MVC (Model-View-Controller)** architecture with **REST APIs**.

- **Model** → Database & Business Logic
- **Controller** → API logic & request handling
- **View** → Frontend (React.js dashboards)

Backend can be implemented using **Laravel (PHP)** or **Node.js (Express)**. This document is backend-agnostic but structured for both.

2. MVC Module Structure

2.1 Core Modules

1. Authentication Module
 2. User (Employee) Management Module
 3. Task / Work Management Module
 4. Performance & Growth Module
 5. Dashboard & Reports Module
 6. Admin Management Module
-

3. Controllers (API Layer)

3.1 AuthenticationController

Purpose: Login, logout, token handling

APIs: - POST `/api/login` - POST `/api/logout` - GET `/api/profile`

3.2 AdminController

Purpose: Admin-only operations

APIs: - GET /api/admin/dashboard - GET /api/admin/employees - POST /api/admin/employee - PUT /api/admin/employee/{id} - DELETE /api/admin/employee/{id}

3.3 EmployeeController

Purpose: Employee self-data access

APIs: - GET /api/employee/dashboard - GET /api/employee/profile - PUT /api/employee/profile

3.4 TaskController

Purpose: Work assignment & tracking

APIs: - POST /api/admin/task (Admin assigns work) - GET /api/admin/tasks - GET /api/employee/tasks - PUT /api/employee/task/{id}/status

3.5 PerformanceController

Purpose: Growth & performance tracking

APIs: - POST /api/admin/performance - GET /api/admin/performance/{employee_id} - GET /api/employee/performance

3.6 ReportController

Purpose: Analytics & reports

APIs: - GET /api/admin/reports/daily - GET /api/admin/reports/monthly - GET /api/admin/reports/yearly

4. Routes Structure

4.1 Public Routes

POST /api/login

4.2 Admin Routes (Role: Admin)

```
GET    /api/admin/dashboard
GET    /api/admin/employees
POST   /api/admin/employee
PUT    /api/admin/employee/{id}
DELETE /api/admin/employee/{id}

POST   /api/admin/task
GET    /api/admin/tasks

POST   /api/admin/performance
GET    /api/admin/performance/{employee_id}

GET    /api/admin/reports/*
```

4.3 Employee Routes (Role: Employee)

```
GET    /api/employee/dashboard
GET    /api/employee/profile
PUT    /api/employee/profile

GET    /api/employee/tasks
PUT    /api/employee/task/{id}/status

GET    /api/employee/performance
```

5. Models (Database Layer)

5.1 User Model

- id (PK)
- name
- email
- password
- role (admin / employee)
- status
- created_at

Relations: - User (Employee) → hasMany Tasks - User (Employee) → hasMany Performances

5.2 Task Model

- id (PK)
- title
- description
- assigned_to (FK → users.id)
- status (pending, in_progress, completed)
- deadline
- created_at

Relations: - Task → belongsTo User - Task → hasOne Performance

5.3 Performance Model

- id (PK)
- employee_id (FK → users.id)
- task_id (FK → tasks.id)
- rating
- feedback
- score
- created_at

Relations: - Performance → belongsTo User - Performance → belongsTo Task

6. Database Relationships (Summary)

- **Users (1) → (M) Tasks**
 - **Users (1) → (M) Performance Records**
 - **Tasks (1) → (1) Performance**
-

7. Middleware & Security

Middleware

- auth (JWT / Sanctum)
- role:admin
- role:employee

Rules

- Employees can access only their own records
 - Admin bypasses ownership checks
-

8. Growth Calculation Logic (Example)

- Completion Rate = Completed Tasks / Assigned Tasks
 - Growth Score = (Completion Rate × 0.6) + (Avg Rating × 0.4)
-

9. Frontend (View Layer)

- React.js
 - Separate Admin & Employee layouts
 - Role-based routing
 - Charts for growth visualization
-

10. API Count Summary

Module	Controllers	APIs
Auth	1	3
Admin	1	6
Employee	1	3
Task	1	5
Performance	1	3
Reports	1	3
Total	6 Controllers	26 APIs

11. Conclusion

This MVC-based technical design ensures scalability, security, and clean separation of concerns. Admin has full control, while employees have restricted, secure access to only their own data.

Document Version: 1.0 **Audience:** Technical / Development Team